Claude Code in Action

Tool Use · Context · Hooks · MCP · SDK

mostly Opus 4.6

Course Overview

What is Claude Code?

- Coding assistants
- Tool use mechanism
- Claude Code demo

Getting Hands On

- Setup & project init
- Adding context
- Making changes
- Context control
- Custom commands
- MCP servers
- GitHub integration

Hooks & SDK

- Hook architecture
- PreToolUse / PostToolUse
- Implementing hooks
- Gotchas & recipes
- The Claude Agent SDK

What Is a Coding Assistant?

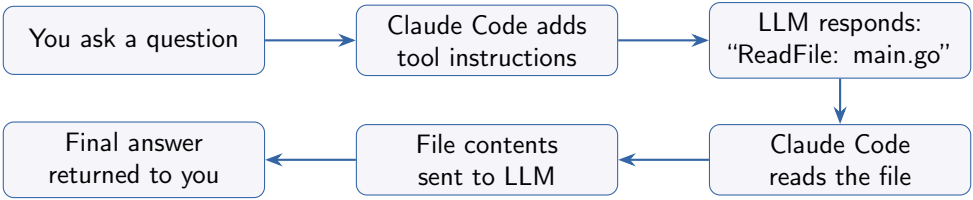
A coding assistant uses language models to tackle complex programming tasks.

How it works (same as a human developer):

1. **Gather context** — read files, understand errors, find relevant code
2. **Formulate a plan** — decide on a fix or implementation strategy
3. **Take action** — edit files, run commands, verify results

Steps 1 and 3 require the model to *interact with the outside world*.
But language models can only process text and return text...

The Tool Use Mechanism



Why Claude's tool use matters

Strong tool use ⇒ tackles harder tasks, extensible platform, better security (no full-codebase indexing).

Setup & Project Initialization

Installation:

```
npm install -g @anthropic-ai/claude-code
cd your-project
claude # launches Claude Code
```

The `/init` command:

- Analyzes your entire codebase
- Creates a `CLAUDE.md` summary (architecture, commands, patterns)
- Included in *every* request — persistent system prompt

Tip

Press **Shift+Tab** to cycle permission modes (e.g. auto-accept edits for the session).

Permissions

Three tool approval tiers:

Tool type	Example	Approval
Read-only	File reads, Grep	Automatic
Bash commands	Shell execution	Per command
File modification	Edit, Write	Per session

Permission modes (cycle with **Shift+Tab**):

- `default` — ask each tool
- `acceptEdits` — auto-approve edits
- `plan` — read-only
- `auto` — smart auto-approve
- `bypass` — skip all prompts

Pre-approve in `settings.json`:

```
"permissions": { "allow": ["Bash(npm run *)"],
                 "deny": ["Read(.env)"] }
```

Evaluation order: **deny** > ask > allow (deny always wins).

Adding Context: CLAUDE.md

Three locations, three scopes:

File	Scope	Shared?
`CLAUDE.md`	Project	Yes (committed)
`CLAUDE.local.md`	Project	No (personal)
`~/.claude/CLAUDE.md`	All projects	No (global)

Memory mode with #:

```
# Use comments sparingly. Only comment complex code.
```

Claude merges this instruction into your `CLAUDE.md` automatically.

File mentions with @:

```
How does the auth system work? @auth
```

Includes the file's contents in the request. Also works inside `CLAUDE.md`.

Making Changes

Key techniques:

- **Screenshots** — paste images directly; Claude reads error messages, UI issues
- **Planning mode** — one of the `Shift+Tab` modes; Claude outlines steps before executing
- **Adaptive thinking** — Claude adjusts reasoning depth; tune with `/effort`
- **Auto mode** — classifier auto-approves safe operations, blocks risky ones

Workflow

1. Describe the problem or feature
2. Let Claude explore, plan, and implement
3. Review the diff, approve or redirect

Controlling Context

Interrupt & Redirect

- **Escape** — stop mid-response, redirect
- **Escape** $\times$ **2** — rewind to earlier point
- Combine with # to fix repeating errors

Commands

- `/compact` — summarize history, keep knowledge
- `/clear` — fresh start, no prior context
- Auto-compaction at $\sim$95% context capacity

When to use what

`/compact` when switching to a *related* task (preserves context).
`/clear` when switching to an *unrelated* task (avoids confusion).

Custom Commands

Create reusable commands:

1. Create `.claude/commands/audit.md`
2. Filename becomes the command: `/audit`
3. Restart Claude Code

Commands with arguments (`$ARGUMENTS`):

```
Write comprehensive tests for: $ARGUMENTS

Testing conventions:
* Use Vitest with React Testing Library
* Place test files in __tests__/ next to source
* Name test files as [filename].test.ts(x)

Coverage: happy paths, edge cases, error states
```

Usage:

```
/write_tests the use-auth.ts file in the hooks directory
```

MCP Servers

Model Context Protocol — extend Claude with new tools.

Example: Playwright (browser control)

```
claude mcp add playwright npx @playwright/mcp@latest
```

Pre-approve permissions in `.claude/settings.local.json`:

```
{
  "permissions": {
    "allow": ["mcp__playwright"],
    "deny": []
  }
}
```

Ecosystem: database interactions, API testing, cloud integrations, file operations, dev tool automation...

Deferred loading: MCP tools lazy-load via `ToolSearch`, keeping unused tool schemas out of context.

GitHub Integration

Run `/install-github-app` in Claude Code.

Two workflows added to `.github/workflows/`**:**

1. **Mention Action** — `@claude` in issues/PRs → Claude analyzes & responds
2. **PR Review Action** — auto-reviews every pull request

Customization options:

- `custom_instructions:` — project-specific context
- `mcp_config:` — attach MCP servers (e.g. Playwright)
- `allowed_tools:` — explicit tool permissions (required in CI)

Hooks: Architecture

PreToolUse

- Runs *before* tool call
- Can **block or rewrite** the operation
- Access control, validation

PostToolUse

- Runs *after* tool call
- Cannot block (already done)
- Formatting, testing, logging

Hook types: `"command"` (shell) | `"http"` (POST JSON to URL)
Config locations: `~/.claude/settings.json` (global) | `.claude/settings.json` (team) | `.claude/settings.local.json` (personal)

Hook Configuration

PreToolUse example — guard specific tool calls:

```
"PreToolUse": [{
  "matcher": "Read|Grep",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/read_hook.js"
  }]
}]
```

PostToolUse example — run formatter after edits:

```
"PostToolUse": [{
  "matcher": "Write|Edit|MultiEdit",
  "hooks": [{
    "type": "command",
    "command": "node ./hooks/format_hook.js"
  }]
}]
```

Implementing a Hook: Protect `.env`

```
async function main() {
  const chunks = [];
  for await (const chunk of process.stdin) {
    chunks.push(chunk);
  }
  const toolArgs = JSON.parse(
    Buffer.concat(chunks).toString()
  );

  const readPath =
    toolArgs.tool_input?.file_path
    || toolArgs.tool_input?.path || "";

  if (readPath.includes('.env')) {
    console.error("You cannot read the .env file");
    process.exit(2); // exit code 2 = block
  }
}

main();
```

Hook receives tool call data via `stdin` as JSON. Exit code 2 blocks the operation with an error message back to Claude.

Hook Gotchas & Practical Recipes

Common Gotchas

- Use **absolute paths** in hook commands
- Restart Claude Code after changing hooks
- Test hooks with simple cases first

Useful hook ideas:

- **TypeScript type-checking** — run `tsc` after edits, feed errors back
- **Auto-formatting** — run Prettier/Black after writes
- **Query duplication prevention** — detect duplicate SQL/API calls
- **Test runner** — auto-run tests on changed files
- **Sensitive file protection** — block access to credentials

Other hook events: Notification, Stop, SubagentStop, PreCompact, UserPromptSubmit, SessionStart, SessionEnd.

The Claude Code SDK

Claude Agent SDK — run Claude Code programmatically (TypeScript, Python, CLI).

```
import { query } from "@anthropic-ai/claude-code";

const prompt = "Look for duplicate queries in ./src/queries";

for await (const message of query({ prompt })) {
  console.log(JSON.stringify(message, null, 2));
}
```

Permissions: read-only by default. Enable writes with:

```
for await (const message of query({
  prompt,
  options: { allowedTools: ["Edit"] }
})) { /* ... */ }
```

Use cases: Git hooks for code review, CI/CD quality checks, automated documentation, build optimization scripts.

Summary

Core Concepts

- Tool use makes LLMs practical
- `CLAUDE.md` = persistent context
- `@` mentions for file focus
- `/compact` vs `/clear`

Power Features

- Custom commands (`$ARGUMENTS`)
- MCP servers (deferred tool loading)
- Hooks (block, rewrite, HTTP)
- Claude Agent SDK
- Auto mode, adaptive thinking
- `/model` to switch models mid-session
- Voice (`/voice`), remote control

Questions?